G.168 Echo Canceller Software User's Guide

Adaptive Digital Technologies, Inc.
February 01, 2019
Issue 6

Copyright © 2009-2019 Adaptive Digital Technologies Inc.
Proprietary

Revision History

1	Adapted from C64X users guide	11/10/2009	BMC
2	Version 11.10	11/21/2011	BMC
3	User guide updated to V11_18	04/30/2017	HFZ
4	User guide updated	06/08/2017	HFZ
5	Update TailLength for LEC	06/16/2017	HFZ
6	Support 1ms frame size	02/01/2019	HFZ

Table of Contents

1. Introduction 1
   1.1 Background on Echo Cancellers 1
      1.1.1 Echo Tail 2
      1.1.2 Adaptive Filtering 3
      1.1.3 Double-Talk 3
      1.1.4 Residual Echo and Nonlinear Processing 3
      1.1.5 Comfort Noise Generation 3
      1.1.6 Stationary Signal Detection 3
   1.2 Echo Canceller Variants 4
      1.2.1 Line Echo Canceller 4
      1.2.2 Network Echo Canceller 4
      1.2.3 Packet Echo Canceller 4
   1.3 Feature Comparison 5
2. Echo Canceller API 5
   2.1 MIPS Conservation 6
   2.2 Object Oriented Interface 7
   2.3 Configuration Parameters 7
   2.4 Software API 13
      2.4.1 Function LEC_ADT_g168Init( ) 13
      2.4.2 Cancel Functions 14
         2.4.2.1 Combined API 14
            2.4.2.1.1 Function LEC_ADT_g168Cancel( ) 14
         2.4.2.2 Split API 15
            2.4.2.2.1 Function LEC_ADT_g168echoCancel() 15
            2.4.2.2.2 Function LEC_ADT_g168postCancel () 16
      2.4.3 Utility Functions 16
            2.4.3.1.1 LEC_ADT_g168GetStatus ( ) 16
         2.4.3.2 LEC_ADT_g168GetConfig ( ) 18
         2.4.3.3 LEC_ADT_g168RuntimeConfig( ) 18
         2.4.3.4 LEC_ADT_g168ChangeChannelNumber( ) 19
         2.4.3.5 LEC_ADT_getSizes 19
      2.4.4 Compile Time Constants 20
      2.4.5 Sample Host Code 21
3. Design / Troubleshooting Considerations 22
4. Additional Information 22

1. Introduction

This document describes Adaptive Digital Technologies' (ADT) G.168 software echo canceller algorithm/software. It focuses on the API, which is common across many target platforms. For information that is specific to a particular target (memory and CPU utilization, deliverable files, etc.) please refer to the appropriate echo canceller data sheet.

The ADT G.168 echo canceller has the following features:

- Ideal for use in VoIP gateway, IP PBX, ATA, and digital network echo cancellers
- Cancels echo caused by 4 to 2 wire conversion circuits that exist in FXS and FXO and in SLIC and DAA circuits
- Cancels multiple reflectors
- G.164/G.165 fax tone disabler
- Stationary signal detector prevents divergence due to tones and other periodic signals
- Tandem free operation
- Robust double-talk detection
- Natural sounding comfort noise generator
- Configurable parameters
- Meets and exceeds ITU G.168 requirements

1.1 Background on Echo Cancellers

Echo in the telephone network is caused by **hybrid** circuits that convert between **two-wire** analog interfaces and **four-wire** analog interfaces, as seen in Figure 1-1 below. The most common device that connects to a 2-wire circuit is the standard analog telephone. The telephone is connected to the telephone central office by a pair of wires. This pair of wires carries speech signals from the telephone to the central office (transmit) and from the central office to the telephone (receive). At the central office, the 2-wire circuit is converted to a 4-wire circuit using a hybrid circuit. The resulting 4-wire circuit uses one pair of wires for each direction of transmission.

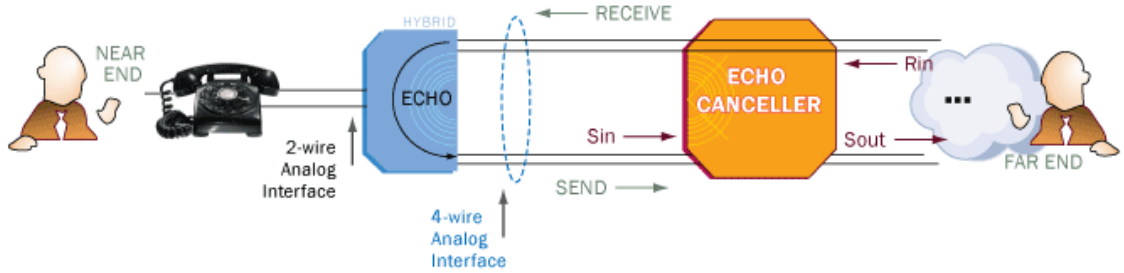

Figure 1-1: Echo Source

A hybrid circuit does its best to prevent from the signal being received via the 4-wire circuit from being reflected and echoed back. Hybrid circuits provide some echo isolation, but not enough when the end-to-end circuit delay is even moderate.

To combat the echo phenomenon, an echo canceller is employed. Today's echo cancellers use sophisticated algorithms running on high speed Digital Signal Processors (DSPs) to combat the echo.

Figure 1-2 below is a block diagram of a telephone circuit, including the echo source and the echo canceller.

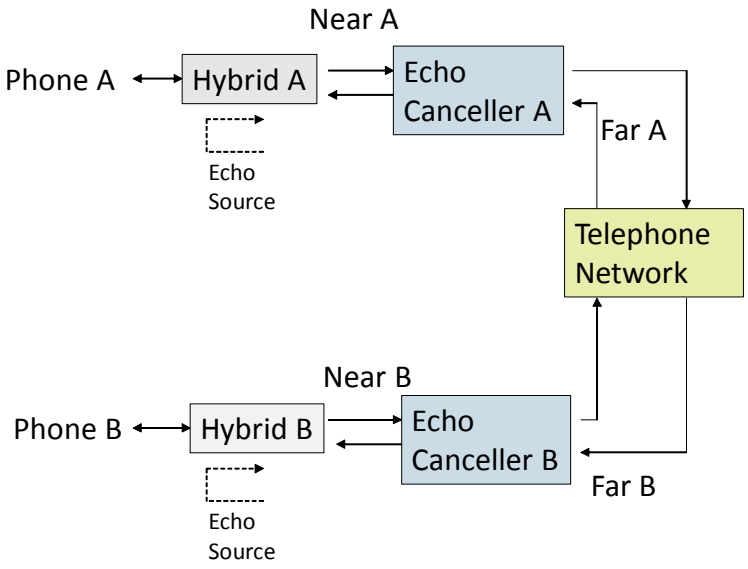

Figure 1-2: Telephone Circuit

Phone A's transmission passes through Hybrid A, Echo Canceller A, the Telephone Network, to remote side's Echo Canceller B and Hybrid B to telephone B. A similar path is established in between Phone B and Phone A.

When Phone A's transmission reaches Hybrid B, part of the signal is reflected by the hybrid back towards Echo Canceller B and therefore back to Phone A. If echo cancellation is not performed (and the network delay is moderate), the speaker at Phone A will perceive his echo.

It is the responsibility of Echo Canceller B to cancel the echo that is induced by Hybrid B. Likewise, it is the responsibility of Echo Canceller A to cancel the echo that is induced by Hybrid A.

The terminology Near End and Far End are usually used when referring to an echo canceller. For example, For the phone A, the Far End signal (from phone B) will enter echo canceller A and passes through unchanged, then is sent out to the hybrid. The hybrid, which is at the Near End with respect to echo canceller A, reflects a portion of the far end signal back towards the echo canceller. The Near End signal received by echo canceller A therefore consists of the sum of Phone A's transmit signal and the echo of the far end induced by Hybrid A.

Now that we have the network topology and terminology described, we need to discuss a few issues that are faced by echo cancellers.

1.1.1 Echo Tail

A hybrid circuit does not create a brick-wall echo. A brick wall echo refers to one where the response of a far end impulse would be an echoed impulse. Since the hybrid is a circuit, the impulse response of the echo path is of a diffuse nature. The impulse response of the hybrid circuit is referred to as the echo tail. The duration of the echo tail is referred as the tail length. An echo canceller must cancel the entire tail.

To make matters more interesting, it is possible that multiple echo sources can be present in the network. This situation is referred to as one with multiple echo tails. A good echo canceller will cancel echo due to all the echo sources in the network.

ADT's echo canceller finds the locations of multiple tails and cancels them.

1.1.2 Adaptive Filtering

As each hybrid circuit is slightly different, each echo tail is different as well. Many factors determine the echo path. It is even possible for an echo tail to change while a circuit is active. This could happen when a second telephone extension is taken off-hook in parallel with the first one.

Due to these variations in echo tails, it is necessary for an echo canceller to adapt to the tail continuously. Adaptive Filtering is employed within echo cancellers to this end. The adaptive filters should converge quickly, but not so quickly that they might diverge under some conditions.
This is especially important when a circuit is first established. The amount of time it takes the echo canceller to adapt to an echo path is referred to as the "convergence time".

1.1.3 Double-Talk

An echo canceller adapts its filter based upon a comparison of the far end signal and the echo of the far end signal that is returned via the near end signal. Ideally, the signal from the near end telephone is quite. If this is not the case, the echo canceller cannot discern between near end speech and echo. The near end speech appears as interference to the filter adaptation algorithm.

The condition where both parties are speaking is referred to as double-talk. If a double-talk condition is not detected properly by the echo canceller, the near end speech will cause the adaptive filter to diverge. It is therefore important to have a reliable double-talk detector.

1.1.4 Residual Echo and Nonlinear Processing

Echo cancellers are not perfect. There is nominally some residual echo that gets past the echo canceller. This residual echo is handled using a non-linear processor. The non-linear processor acts to squelch the residual echo under certain conditions.

1.1.5 Comfort Noise Generation

It is not always good to squelch the residual echo because as a result, near end background noise will be squelched also. This results in the background noise being pulsed on and off as the nonlinear processor activates and deactivates. To avoid this unwanted pulsing, a comfort noise generator is used. Whenever the NLP activates, rather than squelching the signal, a noise signal is generated at the level of the near end background noise.

1.1.6 Stationary Signal Detection

Certain signals, when present over a long period of time, can cause an echo canceller to begin to diverge. These signals include tonal signals, modem signals, and generally any signal that is stationary and correlated. ADT's echo canceller employs a stationary signal detector. When a stationary signal is detected, the echo canceller will temporarily suspend adaptation.

1.2 Echo Canceller Variants

Adaptive Digital's G.168 echo canceller comes in three variants, each of which is designed to suit a specific type of application. The three variants are Line Echo Canceller(LEC), Network Echo Canceller(NEC), and Packet Echo Canceller(PEC).

Note: In the recent version, the Line EC has been merged into Network EC. Customer may use Network EC

1.2.1 Line Echo Canceller

The Line EC (LEC) is designed to operate inside a device that has a local hybrid circuit. The hybrid circuit is the circuit that converts between a4-wire and a 2-wire interface. Hybrid circuits include:

- SLIC (Subscriber Line Interface Circuit)
- FXO (Foreign Exchange – Office)
- FXS (Foreign Exchange – Subscriber)
- DAA (Data Access Arrangement)

Examples of equipment that need a line echo canceller include:

- ATA
- VoIP Gateway with FXO and/or FXS interfaces

The significance of the local hybrid is that there is little delay between the hybrid, the source of the echo, and the echo canceller.

1.2.2 Network Echo Canceller

The Network Echo Canceller (NEC) is designed to operate in a piece of network equipment that may not have an internal echo source. The echo source may be elsewhere in the network. Examples of equipment that may need an echo canceller include:

- VoIP Gateway with digital trunks
- T1/E1 echo canceller

Since the echo source is elsewhere in the network, the network echo canceller must be able to deal with longer echo delays than the line echo canceller. It must also be able to handle signaling tones that are found in the telephone network. It may also need to cancel reflections caused by more than one hybrid circuit in the network. Finally, the network echo must be able to deal with echo-free circuits or circuits in which the echo is already cancelled at the source.

1.2.3 Packet Echo Canceller

The conventional wisdom tells us that an echo canceller should be as "close to" the echo source (hybrid) as possible. For a variety of reasons, this makes the echo canceller perform better. But the introduction of the Voice –Over-IP network, the conventional wisdom has been turned on its side. The added delay incurred by the packet network can cause echo that is not otherwise perceptible to become a problem. This may include not only echo due to hybrid circuits but also some residual acoustic echo from handsets or hands-free devices. As a result, there could be equipment in the field that did not require echo cancellation before VoIP entered the picture.
Since it is not feasible to modify all such equipment, it is sometimes necessary for an echo canceller in a packet network to cancel echo whose source is on the opposite end of the packet network. A packet network echo canceller must be able to handle excessively long echo delays. It must also be able to operate well whether there is echo or not. In fact, it is far more common that a connection be echo free

because the echo is cancelled at the source. Furthermore, a packet echo canceller must be able to deal with the presence of speech compression algorithms within the VoIP network.

Examples of equipment that may need a packet echo canceller include:

- VoIP Gateway
- VoIP Conference Server

1.3 Feature Comparison

The following table compares the echo canceller variants in terms of features.

Feature	G.168 NEC(G.168 LEC)	G.168 Plus™ Packet EC
Maximum Tail Length	NEC max 128 msec LEC max 16 msec	512 msec
Tandem Free Operation	X	X
High Efficiency Tandem Free Operation		X
Packet Loss Concealment		X
Dynamic NLP(reserved)	X	X
Comfort Noise Generator	X	X
Rapid Convergence	X	X
Automatic Tail Search	X	X
Cancels multiple reflectors	X	X
Convergence Monitor	X	X
Tone Disabler	X	X
SS7 Tone Detection	X	X
Double-talk Detection	X	X
Stationary Signal Detector	X	X
Pre-NLP Signal Available	X	X
Split Pre- and Post- Processing	X	X
Handles Residual Acoustic Echo	X	X
eXpress DSP™ compliant	X	X
Functions are “C” callable	X	X
User Configurable Parameters	X	X

2. Echo Canceller API

ADT’s G.168 echo canceller is designed using object oriented methodology to achieve a multiple channel architecture. Originally, ADT’s echo canceller was geared towards static memory allocation and traditional “C” function interfaces. To become compliant with TI’s xDAIS (Express DSP Algorithm Interface Standard), ADT has enhanced the echo canceller to support dynamic memory allocation as well as TI’s IALG interface. ADT’s traditional interface is a subset of the xDAIS code. The traditional interface can be used with no additional CPU or memory overhead. The traditional interface is described in this section.

2.1 MIPS Conservation

In some systems, it may be desirable to make a tradeoff between echo canceller convergence speed and channel density. This can be done by reducing echo canceller CPU utilization (MIPS) in such a way that no echo canceller channel is starved. The way it is accomplished is via a "rolling brown-out" at times when the active channel count is approaching the processor's limit.

By "rolling brown-out," we mean that during each frame, some echo canceller channels will reduce their CPU utilization. But the set of selected channels changes from frame to frame. The effect is to reduce the total peak CPU utilization across all channels.

If a system supports multiple frame sizes using one thread for each frame size, the conservation operates on a per-frame-size basis. Echo cancellers channels are therefore divided into groups, each group having a given frame size.

In order to make this work, each echo canceller in a frame-size group is assigned a unique channel number from 0 to N. The numbering should be contiguous. Each channel is made aware of a running frame count as well as the amount of MIPS reduction that is requested. Based upon this information, the echo canceller channel knows, on a frame-by-frame basis, when it is time to conserve CPU cycles.

The host processor must do the following.

1. Declare and initialize a MIPSConserve structure for each frame-size group. (See g168_user.h, MIPSConserve_t). The initial frame count is arbitrary. Only the Reduction Amount is relevant.
2. Pass a pointer to the MIPSConserve structure to each channel in the frame-size group via the G168ParamsV11_t structure during the call to LEC_ADT_g168Init.
3. Assign channel numbers to each echo canceller channel via the G168ParamsV11_t structure during the call to LEC_ADT_g168Init.
4. Increment the frame count in the MIPSConserve structure once per frame.

The MIPS reduction amount can be changed at any time by modifying the value in the structure. Typically, the reduction amount is set to zero when the active channel count is below X% of the system limit. When the active channel count exceeds X%, the reduction amount may be changed to 1 or more, depending upon the amount of reduction the application requires. Note that as the reduction amount increases, so does the convergence time and reconvergence time. And as the reduction amount increases beyond a certain point, the CPU cycle savings becomes smaller and smaller (diminishing returns.)

Note that it is best if each channel in the frame-size group has a unique channel number and the channel numbers are contiguous. The LEC_ADT_g168ChangeChannelNumber function is provided to enable the host application to change the channel number on the fly. This is not strictly necessary because the time that MIPS reduction is important is when most channels are active, and in that case, the distribution of channel numbers will be pretty uniform. So it may not be worth the extra complexity required to maintain a unique contiguous set of channel numbers.

Here is an example of a system that handles two frame sizes (80 and 160 samples, for example).

```
#define REDUCTION_AMOUNT 2

MIPSConserve_t MIPSConserve80[] = {0, 0};
MIPSConserve_t MIPSConserve160[] = {0,0};
//Whenever a channel is brought up or down:
```

```
if (ActiveChannelCount > FRAME_COUNT_THRESHOLD) {
        MIPSConserve80.ReductionAmount = REDUCTION_AMOUNT;
        MIPSConserve160.ReductionAmount = REDUCTION_AMOUNT;
}
else {
        MIPSConserve80.ReductionAmount = 0;
        MIPSConserve160.ReductionAmount = 0;
}
// Once per frame

FrameXXData.FrameCount += 1;
```

2.2 Object Oriented Interface

The G.168 echo canceller software is designed in an object oriented fashion to simplify multi-channel operation. It is the responsibility of the host software to allocate the persistent (non-volatile) data used by each instance (channel) of the echo canceller. There are three chunks of memory that the echo canceller requires for each instance.

- Instance Structure
- Echo Path Array
- State Structures

2.3 Configuration Parameters

Table 3-1below describes the echo canceller configuration parameters.

Configuration Parameters			
Parameter	**Description**	**Recom-mended Value**	**EC Var-iant**
APIVersion	Specifies the API version that is being used. The constant ADT_EC_API_VERSION is defined in g168_user.h. The reason for specifying the version is to reduce the chance that there is a mismatch between the g168_user.h version and the library version.	ADT_EC_API_VERSION	ALL
ChannelSizeI8	The size, in bytes, of the EC channel structure.	sizeof(G168ChannelInst_t)	ALL
EchoPathSizeI8	The size, in bytes, of the Echo Path array. The value can be set (TapLength+FrameSize+1) * sizeof(ADT_Int16)	sizeof(ADT_Int16) * (MAX_TAP_LENGTH+ MAX_FRAME_SIZE+ 1)	ALL
DAStateSizeI8	The size, in bytes, of the DA State Structure.	Sizeof(G168_DA_State_t)	ALL
SAState	The size, in bytes, of the SA State structure.	Sizeof(G168_SA_State_t)	ALL
SAScratchSizeI8	The size, in bytes, of the SA Scratch Structure.	Sizeof(G168_Scra	ALL

		tch_t)	
BGEchoPathSizeI8	The size, in bytes, of the BGEchoPath Array. The value can be set (TapLength+FrameSize+1) * sizeof(ADT_Int16)	sizeof(ADT_Int16) * (MAX_TAP_LENGTH/4+ MAX_FRAME_SIZE+ 1)	NEC, PEC
DAScratchSizeI8	The size, in bytes, of the DA Scratch Structure	Sizeof(G168_DAScratch_t)	ALL
FrameSize	Number of samples per frame. (Size of Near and Far arrays) **The suggested value should be multiple of 8, for example, 8, 16, 40, 80, etc.**	Application Dependent	All
TapLength	The maximum tail length that the EC will cancel, specified in number of samples. At 8 kHz sampling rate, we have 8 samples/ millisecond. **The suggested value should be multiple of 128(16ms). For example,** **for LEC, 128(16ms)** **for NEC = 512(64ms), 640(80ms), (768)96ms, … 1024(128ms).**	Application Dependent	All
NLPType	The Nonlinear Processor (NLP) performs this additional suppression on the echo residue, by either muting the output signal or generating comfort noise in order to avoid transitions between silence and background noise. Different NLPType indicates different comfortable noise generator(CNG) or combinations. 0= NLP_OFF 1= NLP_MUTE, mute near end in case of NLP 2= NLP_RAND_CNG The NLP can be configured to generate random white noise as CNG, which is a bit harsh sounding 3 = NLP_HOTH When the NLP is engaged, NLP will generate Hoth characteristic of typical background noise as comfortable noise. 4 = NLP_SUPP When the NLP is engaged, NLP will not use CNG, instead it attenuates the residual by the suppression amount NLPMaxSupp dB. 5 = RESERVED BY ADT 6 = NLP_SUPP_AUTO NLP will mix Hoth noise and echo residue, try to make the comfort noise sound natural.	6	All

AdaptEnable	This parameter control the EC FIR's adapatation. 0 = Disable coefficient adaptation 1 = Enable coefficient adaptation	1	All
BypassEnable	0 = Normal Operation 1 = Bypass the echo canceller 2 = Bypass the echo canceller but still adapt its internal states and coefficients	0	All
G165DetectEnable	0 = Disable G.165 Modem tone detector 1 = Enable G.165 Modem tone detector	1	All
DoubleTalkThres	Double talk detector's threshold A condition of double-talk is defined as both parties (far and near end) talking at the same time. The EC's adaptive filter adapts best when the near end input is purely echo in the absence of a near end speaker speaking. When the near end speaker is speaking, that signal is distortion as far as the EC is concerned. If the near end level becomes significantly large with respect to that of the far end, the echo canceller can diverge. A double-talk detector is used to decide when a double-talk condition exists for the purpose of temporarily halting EC adaptation. The double-talk threshold is used to decide when that is the case. Specifically, if the ratio of far end to near end signal is less than the double-talk threshold, double-talk is declared. This parameter is specified in dB. For example, if the Near signal amplitude is 2000, and Far signal amplitude is 4000, then DoubletlakThres = 20 Log (4000/2000) = 6	6	All
MaxDoubleTalkThres	Double talk detector's soft-threshold. This parameter helps prevent false doubletalk detections from occurring by making the doubletalk detector less aggressive. The default value of 40 dB was determined to provide the best performance over a variety of doubletalk scenarios. ADT highly recommends using the default setting.	40	All
SaturationLevel	Some analog circuitry exhibits nonlinear behavior (saturation, compression, clipping) at high signal levels. Echo cancellers do not deal well with nonlinear echo paths. The Saturation Threshold, expressed in dBm, is a threshold above which there may be excessive nonlinearity. For well-designed circuitry, this should be set to **+3 dBm**, which is the maximum digital signal level of 16bit signed integer. If the saturation threshold needs to be set below –6 dBm, there is something seriously wrong with the analog circuitry. **Note**: Adaptive Digital will use 3dBm as the maximum power of the digital signal (-32768 to 32767). The 16bit integer will be treated as Q15 number, normalized to 1. For the Sine wave with amplitude 1, the power is compensated to 3, **3+**20 Log (1) = 3. Customer may choose other equivalent power measurement.	3	All
DynamicNLPAggressiveness	Reserved. Default: -1 disables the dynamic NLP	DYNAMIC_NLP_DISABLE	All

NLPThres	Nonlinear processor threshold. The echo cancellers nonlinear processor engages when the ratio between the far end signal level and the residual signal exceeds a threshold. This condition is indicative that there is little or no near end input speech, only residual echo. Under these circumstances, the NLP engages. The NLPThres defines the threshold for making this decision. This parameter is specified in dB. Range: 0 to 40 Bigger value usually means less aggressive NLP.	24	All
NLPUpperLimitThresConv	Dynamic NLP can hit a temporary threshold as low as 0 dB. NLPUpperLimitThresConv allows users to limit NLP threshold under converged conditions to get less aggressive NLP. This parameter is specified in dB. Range: 0 to 40 Bigger value usually means less aggressive NLP.	18	All
NLPUpperLimitThresUnConv	NLPUpperLimitThresUnConv allows users to limit NLP threshold under un-converged conditions to get less aggressive NLP. This parameter is specified in dB. Range: 0 to 40 Bigger value usually means less aggressive NLP.	12	All
NLPSaturationThreshold	This NLP threshold, specified in dB, is used during periods when a possible saturation condition is detected. (See SaturationThreshold)	6	All
NLPMaxSupp	NLPMaxSupp is only used when NLPType = NLP_SUPP. NLPMaxSupp is the suppression scale, which indicates a limit how many dB the NLP will suppress on the echo residue. This parameter is specified in dB. Range: 0 -90	10	All
CNGNoiseThreshold	Maximum near end noise level used to compute CNG level. If the NLP is set to generate comfort noise, this number defines the maximum comfort noise level. This parameter is specified in dBm. For example, CNGNoiseThreshold is set to 43 means that the maximum comfort noise level is –43 dbm. Range: 0 -90	43	All
AdaptLimit	Maximum number of FIR adaptations per frame. At most, the EC's adaptive filter can be adapted once per sample time, or equivalently Frame Size times per frame. AdaptLimit defines the maximum number of times the EC is allowed to adapt per frame. The recommendation is set nominally to ½ the frame size. Range: 2 – FrameSize Larger value means faster convergence, more CPU cycles.	FrameSize/2	All

	Note: Please set AdaptLimit to even number, this will help algorithm run more efficiently.		
CrossCorrLimit	Maximum number of background FIR adaptations per frame. At most, the EC's adaptive filter can be adapted once per sample time, or equivalently Frame Size times per frame. CrossCorrLimit defines the maximum number of times the background adaptation is allowed to per frame. This is nominally (FrameSize/7+1). Range: 2 – FrameSize Larger value means faster convergence, more CPU cycles. Note: Please set CrossCorrLimit to even number, this will help algorithm run more efficiently.	FrameSize/7+1	NEC, PEC
FIRTapCheckPeriod	Maximum number of secondary adaptation per frame. The default value should be set to 80.	80	NEC, PEC
NFIRSegments	Number of independent FIR Segments (independent echo tails) This parameter is a carryover from our network echo canceller. In the network EC, it is possible to see a sparse tail. In other words, the tail has sections of non-zero energy separated by sections of zero energy. Rather than work on the whole tail, the EC finds the non-zero sections and only cancels those sections. NFIRSegments defines the maximum number of non-zero tail sections that the EC will find and work on. Range: 1, 2, 3 For NEC and PEC, NFIRSegments should be set to 3. For LEC, NFIRSegments MUST be set to 1.	3	
FIRSegmentLength	FIRSegmentLength defines the maximum length (in samples) of each FIR Segment. The value should be multiple of 8, such as 8, 16, 32, ……, etc. Range: 8 to TapLength For NEC and PEC setting, FIRSegmentLength should be set to 48. For LEC, FIRSegmentLength MUST be set to equal to TapLength.	48	NEC, PEC
TandemOperationEnable	When two or more echo cancellers are "faced" toward the same echo source, the "closest" one to the hybrid operates as usual, but any echo cancellers "farther" away can be negatively affected by the nearer echo cancellers. This feature, when enabled, reduces the negative effect. The user may consider using this feature when the echo canceller is placed in network equipment.	0	NEC, PEC

MixedFourWireMode	A network echo canceller can be faced with situations where it is enabled on a circuit that has no echo source such as a four-wire circuit or a circuit connected to a digital phone. In such situations, it is possible that an echo canceller will create a small amount of echo rather than do nothing at all. This feature, when enabled, mitigates this unwanted effect.	0	NEC, PEC
ReconvergenceCheckEnable	Flag to enable/disable the check for reconvergance. 2 == enabled, 0 == disabled	2	All
pMIPSConserve	Pointer to the MIPS-conserve structure. Enables the user to trade off between CPU utilization, convergence speed, and re-convergence speed. typedef struct { ADT_UInt8 FrameCount; ADT_UInt8 ReductionAmount; } MIPSConserve_t; FrameCount: Must be incremented by host application once per frame. For example, the frame size is and four EC instances run together. After all four instances finish to process 80 samples, the application will increased the FrameCount. ReductionAmount: 0…4 0 for no MIPS reduction, 4 for maximum MIPS reduction. **Note**: To disable MIPS conservation feature, users may pass Null pointer for this parameter.	MIPS_CONSERVE_OFF	All
ChannelNumber	Unique channel number assigned. This applies only when using MIPS Conservation	0	All
SmartPacketModeSelect	The packet echo canceller can be programmed to reduce the CPU utilization under echo-free or nearly echo-free line conditions. This is extremely useful because most lines fall into these categories. This field takes on three possible self-explanatory values: SMART_PACKET_SELECT_OFF (0) SMART_PACKET_SELECT_PARTIAL_OPTIMIZATION (1) SMART_PACKET_SELECT_MAXIMUM_OPTIMIZATION (2)	0	PEC
SmartPacketBypassERLThresholddB	If SmartPacketMode is turned on, this threshold, specified in dB, dictates the ERL (Echo Return Loss) threshold above which echo canceller will be bypassed.	40	PEC
SmartPacketSuppressERLThresholddB	If SmartPacketMode is turned on, this threshold, specified in dB, dictates the ERL (Echo ReturnLoss) threshold above which suppression will be performed rather than cancellation.	30	PEC
SmartPacketNLPThresh	If the SmartPacketMode is selected and suppression is	24	PEC

olddB	indicated, this threshold, specified in dB, will supercede the other NLP thresholds.		
G165Status	Bit0: G164FarFlag Bit1: G165FarFlag Bit2: G165FarLatched Bit3: G164NearFlag Bit4: G165NearFlag Bit5: G165NearFlagLatched 0 = Flag clear 1 = Flag set	Returned from EC's G165 tone detector	ALL
Other un-listed parameters	Reserved Debug status	Don't care	ALL

Table 2-1: Configuration Parameters

IMPORTANT NOTES FOR CONFIGURING CERTAIN PARAMETERS FOR NEC AND PEC VARIANTS:

Parameters TapLength, FrameSize, NFIRSegments, and FIRSegmentLength affect memory requirements. Structure sizes are based upon worst-case values, which are defined by the user as compiler constants MAX_TAP_LENGTH, MAX_FRAME_SIZE, MAX_FIR_SEGMENTS, and MAX_FIR_SEGMENT_LENGTH.

The following rules must be followed when determining the appropriate values for these parameters and compile-time constants:
PARAMETERS:

1. **MAX_FIR_SEGMENTS and NFIRSegments must never exceed 3**
2. **NFIRSegments must never exceed MAX_FIR_SEGMENTS**
3. **FIRSegmentLength must never exceed MAX_FIR_SEGMENT_LENGTH**
4. **TapLength must never exceed MAX_TAP_LENGTH**
5. **(FIRSegmentLength * NFIRSegments) must be less than TapLength;**

LEC variant is merged in NEC in recent version. We will still use the "EC_VARIANT_NEC" variant.
#define EC_VARIANT EC_VARIANT_NEC
The tail search is automatically turned off when the following condition holds true:
NFIRSegments = 1;
FIRSegmentLength = TapLength;

The TapLength is no more than 128.

2.4 Software API

2.4.1 Function LEC_ADT_g168Init( )

This function is called each time a new circuit is established. This function is only used if channel-by-channel initialization is being employed.

Header: g168_user.h

Prototype

```
ADT_API ADT_UInt8 LEC_ADT_g168Init (
      G168ChannelInst_t *Channel,
      G168ParamsV11_t *Params,
      ADT_Int16 *EchoPath_p,
      G168_DA_State_t *G168_DAState_p,
      G168_SA_State_t *G168_SAState_p,
      G168_Scratch_t *G168Scratch_p,
      ADT_Int16 *BG_EchoPath_p,
      G168_DAScratch_t *G168_DAScratch_p);
```

Inputs

- Channel: Pointer to the G.168 instance structure.
- Params: Pointer to G.168 configuration parameters structure.
  These parameters are described in section 2.3.
- EchoPath_p: Pointer to the Echo Path array
- G168_DAState_p: Pointer to the DA_State structure
- G168_SAState_p: Pointer to the SA_State structure
- G168Scratch_p: Pointer to the structure of scratch variables
- BG_EchoPath_p: Pointer to the Background Echo Path array
- G168_DAScratch_p: Pointer to the structure of dual-access scratch memory

Outputs

None

Returns:

Result code – defined in g168_user.h

Description

LEC_ADT_g168Init initializes an instance of the echo canceller

2.4.2 Cancel Functions

The echo cancel operation can be done using a single function call or using two function calls. Normally it is done with a single function call (combined API) LEC_ADT_g168Cancel. But in some circumstances, it is helpful to separate the adaptive filtering from the nonlinear processing. For example, if AGC or noise reduction is used, a portion, or all the processing of those algorithms should happen in between the adaptive filter and the nonlinear processor. In that case, it is desirable to use the Split API.

2.4.2.1 Combined API

2.4.2.1.1 Function LEC_ADT_g168Cancel( )

Header: g168_user.h

Prototype

```
ADT_API void LEC_ADT_g168Cancel(
      G168ChannelInst_t *Channel,
      ADT_Int16 NearEnd[],
      ADT_Int16 FarEnd[],
      ADT_Int16 Near_PreNLP[]);
```

Inputs

- Instance: Pointer to the G.168 instance structure.
- `FarEnd`: Array of FrameSize samples that is input into the echo canceller
- `NearEnd`: Array of FrameSize samples that is input into the echo canceller.

Outputs

- `NearEnd`: Array of FrameSize echo-cancelled samples
- NearPreNLP: Array of FrameSize samples that is near end residual of the echo canceller before the NLP.

Returns

Nothing

Description

g168Cancel performs the actual echo cancellation function. The near end is assumed contain a far-end echo component, which is cancelled by g168Cancel.

2.4.2.2 Split API

ADT's echo canceller provides split APIs for users to access pre-NLP residual. This could be done by splitting up the above EC function call into two function calls if users choose to do so.

2.4.2.2.1 Function LEC_ADT_g168echoCancel()

Header: g168_user.h

Prototype

```
void LEC_ADT_g168echoCancel (
      G168ChannelInstance_t *Instance,
      short int Near[],
      short int Far[]
      )
```

Inputs

- Instance: Pointer to the G.168 instance structure.
- Near: Array of FrameSize samples that is input into/output of the echo canceller.
- Far: Array of FrameSize samples that is input into the echo canceller

Outputs

- Near: Array of FrameSize samples that is near end residual of the echo canceller before the NLP.

Returns

Nothing

Description

LEC_ADT_ g168echoCancel() performs the echo cancellation function without NLP. The near end is assumed contain a far-end echo component, which is cancelled by LEC_ADT_g168Cancel(), not passing through pos-cancel executions yet.

2.4.2.2.2 Function LEC_ADT_g168postCancel ()

Header: g168_user.h

Prototype

```
void LEC_ADT_g168 postCancel (
      G168ChannelInstance_t *Instance,
      short int Near[],
      short int Far[]
      )
```

Inputs

- Instance: Pointer to the G.168 instance structure.
- Near: Array of FrameSize samples that is input into/output of post-cancel function.
  This buffer holds the output data from LEC_ADT_g168echoCancel() .
- Far: Array of FrameSize samples that is input into the echo canceller

Outputs

- Near: Array of FrameSize samples that is near end output after the NLP.

Returns

Nothing

Description

LEC_ADT_ g168postCancel () performs the post-echo-cancellation actions such as NLP. The near end input is assumed echo-free, which is already cancelled by LEC_ADT_g168echoCancel().

2.4.3 Utility Functions

2.4.3.1.1 LEC_ADT_g168GetStatus ( )

Header: g168_user.h

Prototype

```
void LEC_ADT_g168GetStatus (
      G168ChannelInst_t *pInstance,
      G168StatusV11_t *pStatus_p);
```

Inputs

- `pInstance`: Pointer to the G.168 instance structure.

- pStatus_p: Pointer to the status of echo canceller.

`G168StatusV11_t` is defined as following:

Name	Description
AdaptReport	The number of times during the previous frame that the adaptive filter was updated.
CrossCorrReport	The number of times during the previous frame that the background adaptive filter was updated.
G165Status	Bit0: G164FarFlag Bit1: G165FarFlag Bit2: G165FarLatched Bit3: G164NearFlag Bit4: G165NearFlag Bit5: G165NearFlagLatched 0 = Flag clear 1 = Flag set
ConvergenceStat	A measure of how well the echo canceller is converged. The number has no particular units but it varies between 15 and 32767 with 15 being the most converged and 32767 being completely unconverged.
NLPFlag	When the NLP kicks in, this flag will be set, otherwise cleared.
DoubleTalkFlag	1 if the echo canceller is in a state of double-talk detection, 0, otherwise.
SmartPacketMode	The state of the packet echo canceller. SMART_PACKET_CANCEL – An echo reflector is present and the echo canceller is cancelling SMART_PACKET_SUPPRESS – A faint echo may be present. The echo canceller is performing suppression only SMART_PACKET_BYPASS – No echo has been detected, the canceller is bypassed.
ERL1	Echo Return Loss of the hybrid, measured indirectly. Specified in dB.
ERL2	Echo Return Loss of the hybrid, measured directly. Specified in dB.
ERLE	Echo Return Loss Enhancement currently being achieved by the echo canceller without the assistance of the NLP. Specified in dB.
G168StatusFlags	Bit0: SS7ToneActive Bit1: NarrowbandToneActive Bit2: DoubleTalkActive Bit3: NLPOn 0 = Flag clear 1 = Flag set
G168EventFlags	Bit0: Reconvergence (event has occurred)
ReflectorLocation[3]	NEC, PEC only. The location(s) of the echo reflectors, specified in samples of delay
ReflectorLength[3]	NEC, PEC only. The length(s) of the echo reflectors, specified in samples of delay.

Outputs

- `pStatus_p`: Pointer to the returned G168StatusV11_t status of echo canceller

Description

`LEC_ADT_g168GetStatus()` returns the status of echo cancellation function.

2.4.3.2 LEC_ADT_g168GetConfig ( )

Header: g168_user.h

Prototype:

```
void LEC_ADT_g168GetConfig(
     G168ChannelInst_t *pInstance,
     G168ParamsV11_t *pParams);
```

Inputs:

pInstance – pointer to the instance structure
pParams – pointer to the place where the parameter structure will be returned

Outputs

pParams – current echo canceller parameters.

Returns:

Nothing

Description:

Returns the current echo canceller parameter settings.

2.4.3.3 LEC_ADT_g168RuntimeConfig( )

Headers: g168_user.h

Prototype

```
void LEC_ADT_g168RuntimeConfig (
      G168ChannelInst_t *pInstance,
      G168RuntimeParamsV11_t *pParams);
```

Inputs

- `pInstance`: Pointer to the G.168 instance structure.
- pParams: Pointer to the subset of parameters that could be changed during the runtime. The structure type is defined in the g168_user.h, and the parameters members are described in the initialization parameters table.

Outputs

`None`

Description

LEC_ADT_g168RuntimeConfig () is the utility to change the modifiable parameters without calling init function, can be used at any time after initialization. Since not all the echo canceller's parameters can be modified at the runtime, the purpose of this utility is to allow users to change some of the parameters on the fly. The parameters defined in structure G168RuntimeParamsV11_t are the ones that can be updated at runtime.

2.4.3.4 LEC_ADT_g168ChangeChannelNumber( )

This function is used in conjunction with the MIPS conservation feature. As channels are started and stopped, it may be necessary to change the channel number of an existing channel for the purpose of MIPS conservation. This function takes care of that.

Prototype:

```
ADT_FTG void LEC_ADT_g168ChangeChannelNumber(
      G168ChannelInst_t *pInstance,
      ADT_Int16 ChannelNumber);
```

Inputs:

`pInstance – pointer` to instance structure
`ChannelNumber` – New channel number

Outputs:

Returns:

Description:

Changes the channel number for the purpose of MIPS conservation.

2.4.3.5 LEC_ADT_getSizes

Header: g168_user.h

Prototype:

```
ADT_UInt8 LEC_ADT_getSizes(
      G168ParamsV11_t *pInParams,
      G168ParamsV11_t *pOutParams);
```

Inputs:

pInParams – pointer to EC parameters structure

Outputs:

pOutParams – pointer to EC parameters structure. The “SizeI8” parameters are updated based upon the input parameters

Returns:

A result code is returned. The codes are the same ones used by LEC_ADT_g168Init and are define din g168_user.h

2.4.4 Compile Time Constants

When including g168_user.h, the echo canceller's header file, it is necessary to define the echo canceller variant either as a compiler flag or as a #define in your source file. It is also necessary to define LINUX if compiling using gcc under LINUX.

The echo canceller variants are defined in g168_user.h. For convenience, the definitions are EC_VARIANT_NEC, and EC_VARIANT_PEC for network(line), and packet echo canceller respectively. **The variant must match the library that you are using.** The library file name contains the variant (NEC, or PEC).

Furthermore, if you wish to override the default values for MAX_FRAME_SIZE, MAX_FIR_SEGMENTS, MAX_FIR_SEGMENT_LENGTH, or MAX_TAP_LENGTH, you must do so using command line defines or "C" code #defines. If you wish to use the defaults, you must define USE_EC_DEFAULTS. These #defines are important in that they affect the size of the echo canceller's state, scratch, and echo path structures/arrays. These constants should reflect the worst-case values that will be used in your system.

If using the "C" #define method, these definitions must precede the #include "g168_user.h" statement.

Here are some examples:

```
gcc –DLINUX –DEC_VARIANT=EC_VARIANT_NEC –DUSE_EC_DEFAULTS myprogram.c

gcc –DLINUX –DEC_VARIANT=EC_VARIANT_NEC –DMAX_FRAME_SIZE=80 -
DMAX_TAP_LENGTH=1024 –DMAX_FIR_SEGMENTS=3 –DMAX_FIR_SEGMENT_LENGTH=1024
```

OR

```
#define LINUX
#define EC_VARIANT=EC_VARIANT_NEC
#define USE_EC_DEFAULTS
#include "g168_user.h"

#define LINUX
#define EC_VARIANT EC_VARIANT_NEC
#define MAX_FRAME_SIZE 80
#define MAX_TAP_LENGTH 1024
#define MAX_FIR_SEGMENTS 3
#define MAX_FIR_SEGMENT_LENGTH 48
#include "g168_user.h"
```

2.4.5 Sample Host Code

A sample is provided here for convenience:

```
#define EC_VARIANT EC_VARIANT_NEC
#define MAX_TAP_LENGTH (128*8)
#define MAX_FIR_SEGMENTS 3
#define MAX_FIR_SEGMENT_LENGTH 48
#define MAX_FRAME_SIZE 8
#include "g168/include/g168_user.h"

#define FRAME_SIZE MAX_FRAME_SIZE

G168ChannelInst_t ECChannel;
G168_SA_State_t G168SAState;
G168_DA_State_t G168DAState;
G168_Scratch_t G168Scratch;
ADT_Int16 EchoPath[MAX_TAP_LENGTH+1+MAX_FRAME_SIZE];
G168_DAScratch_t G168DAScratch;
ADT_Int16 BG_EchoPath[MAX_TAP_LENGTH_D+1+MAX_FRAME_SIZE];

short int Near[MAX_FRAME_SIZE];
short int FarIn[MAX_FRAME_SIZE];
short int Near_preNLP[MAX_FRAME_SIZE];

static G168ParamsV11_t G168Params =
{
      ADT_EC_API_VERSION,
      0,    //Int16 ChannelSizeI8
      0,    //Int16 EchoPathSizeI8
      0,    //Int16 DAStateSizeI8
      0,    //Int16 SAStateSizeI8
      0,    //Int16 SAScratchSizeI8
      0,    //Int16 BGEchoPathSizeI8
      0,    //Int16 DAScratchSizeI8
      FRAME_SIZE, //FrameSize
      MAX_TAP_LENGTH,
      6,    // NLP Type
      1,    //AdaptEnable
      0,    // BypassEnable
      1,    // Enable G165 tone detector
      6,    //DoubleTalkThresh
      40,   //Maximum value for double-talk threshold
      3,    //SaturationLevel
      DYNAMIC_NLP_DISABLE,    //DynamicNLPAggressiveness
      24,   //NLPThresh
      18,   //NLPUpperLimitThresConv
      12,   //NLPUpperLimitThresUnConv
      6,    // NLPSaturationThreshold
      10,   //NLPMaxSupp
      43,   //CNGLevelThreshold
      FRAME_SIZE/2,     //AdaptLimit
      FRAME_SIZE/7+1,   // CrossCorr Limit
      80,   // FIRTapCheckPeriod
      MAX_FIR_SEGMENTS, // NFIRSegments
      MAX_FIR_SEGMENT_LENGTH, // FIR Segment Length
      0,    //TandemOperationEnable
```

```
        0,    //MixedFourWireMode
        2,    //ReConvergenceCheck
        0,    //&MIPSConserve,
        0,    //Channel Number
        SMART_PACKET_SELECT_OFF, //SmartPacketModeSelect;
        40,   //SmartPacketBypassERLThresholddB
        30,   //SmartPacketSuppressERLThresholddB
        24    //SmartPacketNLPThresholddB;
};

        G168Params.ChannelSizeI8 = sizeof(ECChannel);
        G168Params.SAStateSizeI8 = sizeof(G168SAState);
        G168Params.SAScratchSizeI8 = sizeof(G168Scratch);
        G168Params.EchoPathSizeI8 = sizeof(EchoPath);
        G168Params.DAStateSizeI8 = sizeof(G168DAState);
        G168Params.BGEchoPathSizeI8 = sizeof(BG_EchoPath);
        G168Params.DAScratchSizeI8 = sizeof(G168DAScratch);

        Result = LEC_ADT_g168Init(
                          &ECChannel,
                          &G168Params,
                          EchoPath,
                          &G168DAState,
                          &G168SAState,
                          &G168Scratch,
                          BG_EchoPath,
                          &G168DAScratch);

        LEC_ADT_g168Cancel(
                          &ECChannel,&Near[0],
                          &FarIn[0],
                          &Near_preNLP[0]);
```

3. Design / Troubleshooting Considerations

When problems arise either in the lab during integration or in the field, it is extremely helpful to Adaptive Digital to provide recordings of the echo canceller's inputs (far and near end) and outputs (near end) that show the occurrence of the problem. We therefore strongly recommend that a method is implemented for capturing this data and storing it to a file in some manner. When sending recordings, they should be accompanied by the echo canceller settings (initialization parameters) that were in use at the time.

4. Additional Information

This users guide is intended to provide information about the echo canceller that is common across all target platforms. For target-specific information such as memory usage, processor utilization, deliverable files, etc., please refer to the appropriate echo canceller data sheet.